Manipulação de Exceções em TypeScript

1. O que é uma exceção

Exceção é um mecanismo para sinalizar que ocorreu uma situação anormal durante a execução do programa. Em TypeScript, usa-se `throw` para lançar a exceção e `try/catch/finally` para tratá-la.

Exemplo elementar:

```
function divide(a: number, b: number): number {
  if (b === 0) {
    throw new Error('Division by zero is not allowed')
  }

  return a / b
}
```

Aqui, se `b` for zero, a função interrompe seu fluxo normal e lança uma exceção.

2. Estrutura básica: `try`, `catch` e `finally`

```
function runDivision(): void {
  try {
    const result = divide(10, 0)
    console.log('Result:', result)
  } catch (error: unknown) {
    if (error instanceof Error) {
      console.log('Handled error:', error.message)
    } else {
      console.log('Unknown error')
    }
  } finally {
    console.log('Execution finished')
  }
}
```

Observação importante no modo `strict`:
No `catch`, o valor capturado deve ser tratado como `unknown`. Isso é correto e desejável, porque em JavaScript qualquer valor pode ser lançado com `throw`, não apenas objetos `Error`.

3. Por que usar `unknown` no `catch`

Em TypeScript estrito, não é seguro presumir que o erro tenha a propriedade `.message`. Este código está conceitualmente errado:

```
try {
  throw 'simple text error'
} catch (error) {
  console.log(error.message)
}
```

O problema é que `error` pode ser uma string, número, boolean etc.

A forma segura é fazer **narrowing**:

```
try {
  throw 'simple text error'
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log(error.message)
  } else {
    console.log('Non-Error value thrown:', error)
  }
}
```

4. Criando exceções personalizadas

Em sistemas mais organizados, convém criar classes de erro específicas para representar falhas de domínio ou de aplicação.

```
class InvalidAgeError extends Error {
  constructor(message: string) {
    super(message)
    this.name = 'InvalidAgeError'
  }
}
```

Uso:

```
function registerPerson(name: string, age: number): string {
  if (name.trim().length === 0) {
    throw new Error('Name must not be empty')
  }

  if (age < 0) {
    throw new InvalidAgeError('Age must be greater than or equal to zero')
  }

  return `Person ${name} registered successfully`
}
```

Tratamento:

```
function main(): void {
  try {
    const message = registerPerson('Alice', -5)
    console.log(message)
  } catch (error: unknown) {
    if (error instanceof InvalidAgeError) {
      console.log('Validation error:', error.message)
      return
    }

    if (error instanceof Error) {
      console.log('General error:', error.message)
      return
    }

    console.log('Unexpected failure:', error)
  }
}
```

Esse padrão é valioso porque permite distinguir tipos diferentes de falha.

5. Exemplo mais realista em modo `strict`

Considere uma função que converte texto em número inteiro positivo:

```
class InvalidInputError extends Error {
  constructor(message: string) {
    super(message)
    this.name = 'InvalidInputError'
  }
}

function parsePositiveInteger(value: string): number {
  const trimmedValue = value.trim()

  if (trimmedValue.length === 0) {
    throw new InvalidInputError('Input must not be empty')
  }

  const parsedValue = Number(trimmedValue)

  if (!Number.isInteger(parsedValue)) {
    throw new InvalidInputError('Input must be an integer')
  }

  if (parsedValue <= 0) {
    throw new InvalidInputError('Input must be greater than zero')
  }

  return parsedValue
}
```

Uso:

```
function execute(): void {
  try {
    const quantity = parsePositiveInteger('10')
    console.log('Quantity:', quantity)
  } catch (error: unknown) {
    if (error instanceof InvalidInputError) {
      console.log('Invalid input:', error.message)
      return
    }

    if (error instanceof Error) {
      console.log('Application error:', error.message)
      return
    }

    console.log('Unknown problem:', error)
  }
}
```

6. Exceções em código assíncrono

Em funções assíncronas, exceções também podem ocorrer. Nesse caso, usa-se `async/await` com `try/catch`.

```
async function loadUserName(userId: number): Promise<string> {
  if (userId <= 0) {
    throw new Error('Invalid user id')
  }

  return 'John Doe'
}

async function executeAsync(): Promise<void> {
  try {
    const userName = await loadUserName(0)
    console.log(userName)
  } catch (error: unknown) {
    if (error instanceof Error) {
      console.log('Async error:', error.message)
    } else {
      console.log('Unknown async error:', error)
    }
  }
}
```

7. Boas práticas

Lance objetos `Error`, não strings.

Errado:

```
throw 'File not found'
```

Certo:

```
throw new Error('File not found')
```

Crie erros específicos quando o domínio exigir distinções relevantes.

Por exemplo: `InvalidCredentialsError`, `FileReadError`, `PermissionDeniedError`.

Não use exceções para fluxo comum de negócio.

Exceções devem representar situações anormais, não o comportamento normal da aplicação.

No `catch`, trate o erro como `unknown`.

Isso preserva segurança de tipos e evita suposições indevidas.

Use `finally` apenas quando houver necessidade real de limpeza.

Por exemplo, liberar recurso, fechar conexão ou registrar término de operação.

8. Exemplo completo e pequeno

```
class EmptyTextError extends Error {
  constructor() {
    super('Text must not be empty')
    this.name = 'EmptyTextError'
  }
}

function toUpperCaseText(text: string): string {
  if (text.trim().length === 0) {
    throw new EmptyTextError()
  }

  return text.toUpperCase()
}

function app(): void {
  try {
    const result = toUpperCaseText('hello')
    console.log(result)
  } catch (error: unknown) {
    if (error instanceof EmptyTextError) {
      console.log('Validation failure:', error.message)
      return
    }

    if (error instanceof Error) {
      console.log('Unexpected error:', error.message)
      return
    }

    console.log('Unknown thrown value:', error)
  }
}
```

9. Síntese final

Em TypeScript com `strict`, o tratamento de exceções deve seguir esta lógica:

1. **Lançar** erros com `throw`
2. **Capturar** com `try/catch`
3. **Tratar o valor capturado como `unknown`**
4. **Refinar o tipo** com `instanceof Error` ou com classes de erro personalizadas
5. **Evitar** lançar strings ou usar exceções como fluxo ordinário

Esse estilo produz código mais seguro, mais legível e mais profissional.